

PROJECT DOCUMENTATION

NLP Pipeline

A modular Nepali+English NLP processing pipeline with a plug-and-play module provider system.

Native modules, HuggingFace models, local code, and remote APIs — all normalized into one envelope contract.

TEAM

Bibidh Subedi

Biprash Pandey

Bhim Prasad Upadhaya

Bibek Gautam

Software Engineering class project — Status: Complete · 41/41 tests passing

Table of Contents

1	Project Overview & Quick Start
2	Communication Schema: Envelope & Manifest Contracts
3	Pipeline Configuration: Choosing, Ordering & Uploading Modules
4	Demo Guide: NLP Pipeline with Module Provider System
5	Module Status & Test Report
A	Appendix A: Module Manifests
	A.1 Code provider
	A.2 HuggingFace provider
	A.3 API provider
B	Appendix B: Dependencies

SECTION 1

Project Overview & Quick Start

Modular Nepali NLP pipeline with plug-and-play modules.
Built for Software Engineering class project.

Setup

```
pip install -r requirements.txt
```

Usage

```
python cli.py
```

Type a Nepali sentence, press Enter. The pipeline runs every active module in sequence and shows the result. Type `quit` to exit.

Web UI

```
python run_web.py
```

Then open `http://127.0.0.1:5000` in your browser.

- **Analyze** — run any Nepali/English text through the pipeline with sentiment, NER highlighting, spellcheck suggestions, stage-by-stage audit trail, and the full envelope JSON
- **Modules** — upload a new `.py` module (auto-added to the pipeline), register extension modules from a manifest file or pasted JSON, test them, and remove them
- **Config** — interactively choose which modules run in the pipeline and in what order (enable / disable / reorder); changes persist to `orchestrator/pipeline_config.json` and apply to the next analysis

See [PIPELINE_CONFIG.md](#) for the full design: how module selection works, how uploading new modules works, and how to do it all from the CLI.

Run Tests

```
pytest tests/ -v
# or without pytest:
python run_tests.py
```

Project Structure

modules/	– one file per module, each exposes process(envelope) -> envelope
orchestrator/	– glue code: run_pipeline() and config
utils/	– shared envelope factory, schema validator, logger
tests/	– unit + integration tests
data/	– benchmark sentences
cli.py	– demo REPL
run_web.py	– web UI entry point (Flask)
web/	– Flask app: templates, static assets, pipeline runner

Adding a Module

The pipeline is runtime-configurable now — you pick which modules run and in what order, no code edits needed.

```
python cli.py enable my-module      # add a module to the pipeline
python cli.py disable modules.ner  # remove one
python cli.py move sentiment-code up
python cli.py upload my_module.py  # upload a new module, added automatically
```

Or use the web UI **Config** page. See [PIPELINE_CONFIG.md](#) for the full explanation of the logic.

Team

- Bibidh Subedi
- Biprash Pandey
- Bhim Prasad Upadhaya
- Bibek Gautam

Status

Complete

test

SECTION 2

Communication Schema: Envelope & Manifest Contracts

Two-party architecture



The pipeline does not know or care whether a module came from a local Python file, HuggingFace Hub, or a remote API. The registry + adapter system normalizes all three into the same `process(envelope) -> dict` interface.

Module Manifest (what a provider gives us)

A provider fills out a JSON manifest validated against

`schema/module_schema.json`. Required fields:

Field	Type	Description
<code>module_id</code>	string	Unique snake_case ID, e.g. <code>sentiment-hf</code>
<code>name</code>	string	Human-readable name
<code>version</code>	string	Semantic version <code>major.minor.patch</code>
<code>provider_type</code>	enum	<code>"huggingface"</code> , <code>"code"</code> , or <code>"api"</code>
<code>config</code>	object	Provider-type-specific config (see below)

Optional fields: `description`, `language` (ISO 639-1 codes), `behavior` (`"annotate"` or `"mutate"`), `annotations_key`.

Config by provider type

`code` — provider ships a Python module with `process(envelope)`:

```
{ "entry_point": "modules.sentiment_analyzer" }
```

huggingface — provider specifies a HuggingFace model:

```
{ "model": "cardiffnlp/twitter-xlm-roberta-base-sentiment", "task": "text-classification", "device": "cpu" }
```

api — provider exposes an HTTP endpoint:

```
{
  "endpoint": "http://localhost:5001/sentiment",
  "method": "POST",
  "timeout": 30,
  "response_annotations_field": "sentiment",
  "response_confidence_field": "score"
}
```

The API adapter sends the full envelope as `POST { "envelope": <envelope> }` and reads results back from the response JSON using dot-path field resolution.

Envelope Schema (per-module data contract)

Key	Type	Required	Mutability
<code>pipeline_id</code>	string (UUID)	yes	read-only
<code>original_input</code>	string	yes	read-only (orchestrator enforces this)
<code>current_text</code>	string	yes	read-write — module writes result here
<code>encoding</code>	string	yes	read-only — always <code>"utf-8"</code>
<code>history</code>	list[dict]	yes	append-only — never modify existing entries
<code>metadata.language</code>	string	yes	read-only
<code>metadata.annotations</code>	dict	yes	module writes its own key, never overwrites others
<code>metadata.confidence_scores</code>	dict	yes	module writes its own key
<code>errors</code>	list[dict]	yes	append-only

History entry schema

Each history entry must have: `module`, `input`, `output`, `status`, `timestamp`, `meta`.

`status` is one of: `success`, `failed`, `skipped`.

Use `skipped` (not `success`) when a module receives empty input and does no real work — see `modules/normalizer.py`, `modules/spellcheck.py`, and `modules/ner.py` for the reference pattern.

Error entry schema

Each error entry must have: `module`, `error`.

How to build and check an envelope

```
from utils.envelope_factory import new_envelope
from utils.validate_envelope import validate_envelope

envelope = new_envelope("राम स्कूल जान्छ") # always start here, never hand-build one
result = your_module.process(envelope)

validate_envelope(result) # raises if shape is wrong
```

Run `validate_envelope(your_envelope)` after your function returns. If it raises, the orchestrator will catch it and skip you — better to catch it yourself first.

Compatibility rules

- Do not change `utils/envelope_factory.py`, `utils/validate_envelope.py`, or `utils/logger.py` unless there's an actual schema bug. These are the shared contract — changing them breaks every module.
- If the schema needs to evolve (e.g. add a new field), update `REQUIRED_TOP_KEYS`-equivalent logic in `envelope_schema.json` / `validate_envelope.py` **and** tell every module owner.
- The orchestrator imports `orchestrator.config` and reads `config.ACTIVE_MODULES` fresh on every `run_pipeline()` call — it does not cache the list at import time. If you add a module discovery mechanism, don't break that lookup path.
- `original_input` is read-only. If your module changes it, the orchestrator reverts it and logs an error into `errors` — your module's other work still lands, but you'll see the revert in the log.
- Your module must accept and return the *same* envelope dict. Returning anything else (a different dict, a string, `None`, etc.) causes the

orchestrator to reject your output and log an error instead of using it.

Quick reference: what breaks compatibility

What you do	What breaks
Rename a file in <code>modules/</code>	<code>config.py</code> still references old name → module skipped
Change <code>process(envelope)</code> signature	Orchestrator can't call it → exception → fail-soft skip
Return something that isn't a dict with required keys	<code>validate_envelope</code> raises → module's work discarded
Mutate <code>original_input</code>	Orchestrator detects and reverts it, logs error
Modify existing entries in <code>history</code>	Downstream modules see corrupted history (not enforced, but wrong)
Write to another module's key in <code>metadata.annotations</code>	Overwrites their output silently
Add a test function that takes arguments	<code>run_tests.py</code> and <code>pytest</code> both call <code>fn()</code> → <code>TypeError</code>
Use pytest-only fixtures (<code>monkeypatch</code> , <code>tmpdir</code> , etc.)	Test passes in pytest but fails in <code>run_tests.py</code>
Change <code>utils/validate_envelope.py</code> 's required fields	Existing modules that passed validation now fail

CLI Reference

```
python3 cli.py run <text>           Run full pipeline + registered modules
python3 cli.py run --file <path>    Run on file contents
python3 cli.py list                 List registered modules
python3 cli.py register <manifest.json> Register a module from manifest
python3 cli.py remove <module_id>   Unregister a module
python3 cli.py config               Show pipeline configuration
python3 cli.py demo                 Sentiment analysis demo (REPL)
python3 cli.py interactive           Interactive REPL mode
```

Provider quick-start

1. Code provider (simplest)

Write a Python file with `process(envelope: dict) -> dict`. Register it:

```
python3 cli.py register manifests/sentiment_code.json
```

2. HuggingFace provider

Fill out `manifests/sentiment_huggingface.json` with your model ID.

Requires `pip install transformers torch`.

3. API provider

Deploy your model behind an HTTP endpoint. Fill out

`manifests/sentiment_api.json` with the endpoint URL. The adapter sends `{ "envelope": <envelope> }` and reads results from the response.

Start the mock server for testing:

```
python3 scripts/mock_sentiment_server.py # runs on :5001
python3 cli.py register manifests/sentiment_api.json
python3 cli.py run "This is great!"
```

SECTION 3

Pipeline Configuration: Choosing, Ordering & Uploading Modules

This document explains the logic behind **which modules run in the pipeline**, **how you turn them on/off**, and **how you add brand-new modules** (either by uploading code or registering a manifest).

Everything described here is implemented — the quickest way to see it working is the **Config** page of the web UI (`python run_web.py`).

1. The big idea: one pipeline, two kinds of modules

Every module that can run in the pipeline belongs to one of two kinds, but they all live in **one ordered list**.

Kind	What it is	Identifier	Example
Native module	A Python file in <code>modules/</code> exposing <code>process(envelope) -> envelope</code>	Dotted import path	<code>modules.normalizer</code>
Registry module	A module registered via a JSON manifest (code, HuggingFace, or API provider)	Registry <code>module_id</code>	<code>sentiment-code</code>

The pipeline is just an ordered list mixing both:

```
{
  "steps": [
    "modules.normalizer",
    "modules.spellcheck",
    "modules.ner",
    "sentiment-code"
  ]
}
```

This list is the **single source of truth**. It is persisted to `orchestrator/pipeline_config.json` and is auto-created on the first change (it is git-ignored, so it never pollutes the repo).

2. How "in the pipeline" vs "not in the pipeline" works

There is no per-module boolean flag. **A module is enabled if and only if its identifier appears in the `steps` list.** Position in the list = position in the pipeline.

Operation	What happens
Enable	The id is appended to the end of <code>steps</code> and the file is saved
Disable	The id is removed from <code>steps</code> and the file is saved
Reorder	The id is swapped one position up/down and the file is saved

The orchestrator (`run_pipeline`) reads this list for every run, so a change takes effect on the **next analysis** — no restarts, no code edits.

Defaults (before anything is saved)

If `pipeline_config.json` does not exist, the pipeline defaults to the three core modules:

```
DEFAULT_CORE_MODULES = [
    "modules.normalizer",
    "modules.spellcheck",
    "modules.ner",
]
```

First-run bootstrap: the web UI and CLI auto-register the built-in `sentiment-code` module and enable it *only* when the config file has never been saved — so the demo behaves exactly as before. The moment you save any configuration, your saved list wins and nothing is auto-enabled.

3. How a module gets resolved and executed

`orchestrator/orchestrator.py` resolves each step id at run time:

```
if "." in step:          # dotted -> native module
    module = importlib.import_module(step)
else:                   # bare id -> registry extension
    module = Registry().load(step)  # adapter for code/huggingface/api
```

Both kinds expose the same `process(envelope) -> envelope` contract, so the orchestrator treats them identically: it runs each step in order, rejects non-dict returns, reverts any mutation of `original_input`, validates the envelope shape after every step, and **fails soft** (a broken module is logged into `envelope["errors"]` and the pipeline keeps going).

4. Where the controls live

Web UI (`python run_web.py`)

- **Config page** (`/config`) — the pipeline editor:
- Active pipeline shown in order with ↑ **Up** / ↓ **Down** / **Disable** buttons
- Every available-but-disabled module listed with an **Enable** button
- **Modules page** (`/modules`) — every registered module shows an **in pipeline / not in pipeline** badge plus Enable/Disable, Test, Remove.

CLI

```
python cli.py config                # show pipeline + status
python cli.py enable <module_id>    # add to pipeline
python cli.py disable <module_id>   # remove from pipeline
python cli.py move <module_id> up|down # reorder
python cli.py run "text"            # runs only enabled steps
```

The `<module_id>` is either a dotted native path (`modules.ner`) or a registry id (`sentiment-code`).

5. Adding a brand-new module

There are three ways, all ending with the module appearing on the **Modules** page (and then on the **Config** page as something you can enable).

A. Upload a Python file (web or CLI) — the fastest path

A `.py` file that exposes `process(envelope) -> envelope` is all you need.

Web: Modules → *Upload a new module* → pick the file (optionally give a `module_id`, display name, description) → **Upload & add to pipeline**.

CLI:

```
python cli.py upload path/to/my_module.py --module-id my-module --name "My Module"
```

What happens server-side (this is the whole logic):

1. **Sanitize the id** — must match `^[a-z][a-z0-9_-]*$`; defaults to the file name if not given.
2. **Collision checks** — the id must not already be registered, and `modules/<id>.py` must not already exist.
3. **Syntax check** — the source is `compile()`d before anything is written.

4. **Save** — the file is written to `modules/<id>.py`.
5. **Import check** — the file is imported and must expose a callable `process`; on failure the file is deleted and the upload is rejected.
6. **Auto-manifest** — a `provider_type: "code"` manifest is generated (`entry_point: "modules.<id>"`) and registered in `registry/modules.json`.
7. **Auto-enable** — the id is appended to the pipeline `steps`, so the new module is immediately part of the pipeline. You can disable/reorder it from the Config page.

B. Register a manifest (web or CLI)

For modules that are already importable code, a HuggingFace model, or a remote API:

- **Web:** Modules → *Register a module by manifest* → upload/paste JSON.
- **CLI:** `python cli.py register manifest.json`

The manifest must match `schema/module_schema.json` (e.g. `manifests/sentiment_huggingface.json`). Registered modules appear in the pipeline editor but are **not** auto-enabled — you decide on the Config page (unless you use the upload path, which auto-enables by design).

C. Classic native module

Drop `your_module.py` into `modules/` exposing `process(envelope)` and it automatically appears in the Config page's available list (native modules are discovered by scanning `modules/*.py`). Enable it from the Config page — no need to edit `ACTIVE_MODULES` in code anymore.

6. File reference

Path	Purpose
<code>orchestrator/config.py</code>	<code>PipelineConfig</code> class; <code>ACTIVE_MODULES</code> live list; <code>DEFAULT_CORE_MODULES</code> ; <code>enable/disable/move/set_steps</code>
<code>orchestrator/pipeline_config.json</code>	Persisted <code>{"steps": [...]}</code> (auto-created, git-ignored)
<code>orchestrator/orchestrator.py</code>	<code>run_pipeline(envelope, steps=None, registry=None)</code> — resolves native + registry steps
<code>registry/registry.py</code> + <code>registry/modules.json</code>	Extension module manifests and adapter lifecycle
<code>registry/adapter.py</code> , <code>registry/adapters/*</code>	Adapters that turn manifests into <code>process()</code> objects

Path	Purpose
<code>web/pipeline_runner.py</code>	<code>ensure_builtins()</code> bootstrap, native module discovery, <code>analyze()</code>
<code>web/app.py</code>	Config/Modules routes, <code>.py</code> upload endpoint
<code>modules/</code>	Native module files (core + uploaded)

7. Design decisions worth knowing

- **One list, not two** — the old split (hardcoded core stages + "run every registered extension") made it impossible to choose. A single ordered `steps` list makes "which modules run" and "in what order" one simple question. `ACTIVE_MODULES` still exists as the live in-memory list the orchestrator reads, so the existing integration tests that pin it keep working.
- **Runtime, not code-editing** — the old way to disable a module was to comment out a line in `config.py`. Now a POST from the UI or a CLI command persists the choice, and every subsequent run honours it.
- **Fail soft everywhere** — a missing or broken module is recorded in `envelope["errors"]` with *"module 'X' not found or broken: ..."* and the rest of the pipeline continues.
- **Uploads are code modules** — an uploaded `.py` becomes a `provider_type: "code"` registry entry pointing at itself, so it gets schema validation, testing, removal, and pipeline membership for free.
- **Tests are deterministic** — config file is git-ignored and tests that assert an exact module order pin `ACTIVE_MODULES` to `DEFAULT_CORE_MODULES` first, so a locally-saved pipeline can't break them.

8. Example walkthrough

1. `python run_web.py`
2. Modules -> Upload a new module -> `my_length.py` -> Upload & add to pipeline
(file saved to `modules/my_length.py`, manifest registered, enabled)
3. Config -> `my_length` is at the end of the pipeline
4. Config -> ↑ Up twice, Disable `modules.spellcheck`
5. Analyze a sentence -> history shows normalizer -> ner -> sentiment-code -> `my_length`
6. `python cli.py config` # same state, now persisted to `pipeline_config.json`

SECTION 4

Demo Guide: NLP Pipeline with Module Provider System

What we built

A modular NLP pipeline for Nepali + English text with a **plugin system** that lets anyone plug in their model — whether it's a HuggingFace model, a standalone Python file, or a remote API — through a single JSON manifest.

The whole thing runs from the command line.

How it works (the big picture)



The envelope (the data contract)

Every module receives and returns one dict called the **envelope**:

```
{
  "pipeline_id": "uuid",
  "original_input": "raw text",          # read-only, never changed
  "current_text": "cleaned text",        # modules read/write this
  "encoding": "utf-8",
  "history": [...],                     # append-only log of what ran
  "metadata": {
    "language": "ne",
    "annotations": {                    # each module writes its own key
      "spellcheck": {...},
      "ner": {...},
      "sentiment": {"label": "positive", "score": 0.75}
    },
    "confidence_scores": {}
  },
  "errors": []                          # append-only
}
```

This is defined in `schema/envelope_schema.json`. Modules never touch each other's keys.

The module manifest (the provider contract)

A provider fills out a JSON file like this:

```
{
  "module_id": "sentiment-code",
  "name": "Sentiment Analyzer (Built-in)",
  "version": "1.0.0",
  "provider_type": "code",
  "language": ["en", "ne"],
  "behavior": "annotate",
  "annotations_key": "sentiment",
  "config": {
    "entry_point": "modules.sentiment_analyzer"
  }
}
```

Three provider types, three `config` shapes:

Provider type	Config needs	What the adapter does
<code>code</code>	<code>entry_point</code> (dotted Python path)	<code>importlib.import_module(entry).process(envelope)</code>
<code>huggingface</code>	<code>model</code> (HF Hub model ID), <code>task</code>	<code>transformers.pipeline(task, model=...)</code>

Provider type	Config needs	What the adapter does
api	endpoint (URL)	<code>requests.post(endpoint, json={"envelope": envelope})</code>

The adapter normalizes all three into the same `process(envelope) -> envelope` call. The pipeline never knows the difference.

File structure (what matters)

```

nlp-pipeline/
├─ schema/
│   ├── envelope_schema.json    # envelope contract (pre-existing)
│   └─ module_schema.json      # manifest contract (NEW)
├─ registry/
│   ├── adapter.py              # ABC: all adapters implement this
│   ├── registry.py             # stores manifests, creates adapters
│   └─ adapters/
│       ├── code.py             # imports Python module
│       ├── huggingface.py      # loads HF transformers pipeline
│       └─ api.py               # POSTs to HTTP endpoint
├─ modules/
│   ├── normalizer.py           # cleans text (pre-existing)
│   ├── spellcheck.py           # spell checker (pre-existing)
│   ├── ner.py                  # named entity recognition (pre-existing)
│   └─ sentiment_analyzer.py    # keyword-based sentiment (NEW)
├─ manifests/
│   ├── sentiment_code.json     # manifest for built-in module
│   ├── sentiment_huggingface.json # manifest for HF model
│   └─ sentiment_api.json       # manifest for API endpoint
├─ scripts/
│   └─ mock_sentiment_server.py # fake API server for demo
├─ cli.py                      # full CLI (rewritten)
└─ orchestrator/
    ├── config.py               # pipeline stage order
    └─ orchestrator.py          # runs modules in sequence

```

Demo script (step by step)

Before class

```
# 1. Make sure you're in the project directory
cd ~/se/nlp-pipeline

# 2. Install deps (if not already)
pip install jsonschema requests
```

Demo part 1: Basic pipeline (2 min)

```
# Show the pipeline running on English text
python3 cli.py run "This is an amazing and wonderful day!"

# Show it on Nepali text – notice NER picks up entities
python3 cli.py run "राम काठमाडौं जान्छ"

# Show negative sentiment
python3 cli.py run "The results were terrible and disappointing"

# Show with JSON output for full envelope
python3 cli.py run "I love this beautiful city" --json
```

What to point out:

- Text flows through 4 stages: normalizer → spellcheck → ner → sentiment
- Each stage appends to `history` — full audit trail
- The envelope has `annotations` from each module, keyed by module name
- `original_input` is never modified (provenance preserved)

Demo part 2: The three provider types (3 min)

a) Code provider (already running)

```
# The built-in sentiment is a "code" provider
# Show the manifest
cat manifests/sentiment_code.json

# Show the adapter loading it
python3 -c "
from registry.registry import Registry
reg = Registry()
reg.register({'module_id': 'test-code', 'name': 'Test', 'version': '1.0.0',
            'provider_type': 'code', 'config': {'entry_point': 'modules.sentiment_analyzer'}})
adapter = reg.load('test-code')
print('Adapter type:', type(adapter).__name__)
print('Module loaded:', adapter._module)
"
```

b) API provider

```
# Terminal 1: Start the mock API server
python3 scripts/mock_sentiment_server.py

# Terminal 2: Register and use it
python3 cli.py register manifests/sentiment_api.json
python3 cli.py run "This is terrible and I hate it"

# Show that the pipeline calls out to an HTTP endpoint
# but the output looks identical to the code provider

# Clean up
python3 cli.py remove sentiment-api
```

What to point out:

- The API server is just a Python script using `http.server` — could be anyone's Flask/FastAPI service
- The adapter sends the full envelope as JSON, reads back annotations
- The pipeline doesn't care it went over the network

c) HuggingFace provider (if transformers is installed)

```
python3 cli.py register manifests/sentiment_huggingface.json
python3 cli.py run "This movie was absolutely fantastic!"
python3 cli.py remove sentiment-hf
```

If **transformers** isn't installed, just show the manifest and explain:

```
cat manifests/sentiment_huggingface.json
# "model": "cardiffnlp/twitter-xlm-roberta-base-sentiment"
# Same envelope goes in, same envelope comes out
# Just the adapter loads a real neural network instead of keywords
```

Demo part 3: Registering a new module live (2 min)

```
# Show how anyone can add a module in 30 seconds
# 1. Write a manifest
cat > /tmp/my_module.json << 'EOF'
{
  "module_id": "sentiment-api-live",
  "name": "Live API Demo",
  "version": "1.0.0",
  "provider_type": "api",
  "language": ["en"],
  "behavior": "annotate",
  "annotations_key": "sentiment",
  "config": {
    "endpoint": "http://localhost:5001/sentiment",
    "method": "POST",
    "timeout": 10,
    "response_annotations_field": "sentiment",
    "response_confidence_field": "score"
  }
}
EOF

# 2. Register it
python3 cli.py register /tmp/my_module.json

# 3. Run it
python3 cli.py run "This is great"

# 4. Remove it
python3 cli.py remove sentiment-api-live
```

Demo part 4: CLI overview (1 min)

```
# Show all commands
python3 cli.py --help

# List what's registered
python3 cli.py list

# Show pipeline config
python3 cli.py config

# Show the interactive REPL
python3 cli.py demo

# (type a few sentences, show the colored sentiment output)
```

Key talking points

1. **The schema is the contract.** Module providers don't need to read our code — they just fill out a JSON manifest and either ship code, point to a model, or expose an endpoint. We validate their manifest against `module_schema.json` before anything runs.
2. **Three provider types, one interface.** The adapter pattern means the pipeline is provider-agnostic. A HuggingFace model, a local Python file, and a remote API all get normalized into `process(envelope) -> envelope`. The orchestrator never imports `transformers` or `requests`.
3. **Fail-soft design.** If a module crashes, the error is logged into `envelope["errors"]` and the pipeline keeps going. One bad module doesn't kill the whole run.
4. **Envelope = audit trail.** Every module's input/output/status/timestamp is in `history`. You can trace exactly what happened to any piece of text.
5. **Nepali NLP is hard.** The existing normalizer handles Unicode NFC normalization, zero-width character stripping, and Devanagari-specific punctuation rules. The spellchecker uses Levenshtein distance. The NER uses a gazetteer with longest-match-first scanning. Our sentiment module handles both English and Nepali word lists.

Quick troubleshooting

Problem	Fix
<code>python: command not found</code>	Use <code>python3</code> instead
<code>ModuleNotFoundError: No module named 'jsonschema'</code>	<code>pip install jsonschema requests</code>

Problem	Fix
API adapter fails	Make sure <code>mock_sentiment_server.py</code> is running on port 5001
HuggingFace adapter fails	<code>pip install transformers torch</code> (takes ~2GB)
Tests show 2 failures	Pre-existing bugs in <code>test_envelope_factory</code> and <code>test_orchestrator</code> — not from our changes

SECTION 5

Module Status & Test Report

Updated when code is committed and tests pass — not when someone says "I finished."

Module	Owner	Status
normalizer	Bhim	DONE, committed, 7 unit tests passing
spellcheck	Bhim	DONE, committed, 6 unit tests passing
ner	Bhim	DONE, committed, 8 unit tests passing
orchestrator core	Bhim	DONE, committed — <code>run_pipeline(envelope)</code> , enforces <code>original_input</code> immutability, rejects non-dict returns
schema + shared utils	Bhim	DONE, committed
SCHEMA.md	Bhim	DONE, committed
benchmark_sentences.json (10 entries)	Bhim (seed) / Bibek (ongoing growth)	DONE — seed set committed, Bibek to extend as new capabilities land
test_integration.py (benchmark runner)	Bibek	DONE, committed — auto-picks up new benchmark entries
test_module_integration.py	Biprash	DONE, committed — 7 integration tests
run_tests.py	Bibek	DONE, committed — no-pytest runner, verified against pytest for parity
sentiment (or other external-team module)	Group Y	not started — pending contact, awaiting <code>process(envelope)</code> wrapper

Test totals as of last commit: 41/41 passing via both `pytest tests/ -v` and `python3 run_tests.py`.

Test Suite Conventions

Bibek's unit + integration tests go here.

Convention:

- `test_<module_name>.py` for unit tests of a single module's `process()` function
- `test_pipeline_integration.py` for full end-to-end orchestrator runs

SECTION A

Appendix A: Module Manifests

Reference manifest files shipped with the project. Validated against `schema/module_schema.json` before anything runs.

A.1 Code provider — `manifests/sentiment_code.json`

```
{
  "module_id": "sentiment-code",
  "name": "Sentiment Analyzer (Built-in)",
  "version": "1.0.0",
  "provider_type": "code",
  "description": "Keyword-based sentiment analysis for English and Nepali. No ML required.",
  "language": ["en", "ne"],
  "behavior": "annotate",
  "annotations_key": "sentiment",
  "config": {
    "entry_point": "modules.sentiment_analyzer"
  }
}
```

A.2 HuggingFace provider — `manifests/sentiment_huggingface.json`

```
{
  "module_id": "sentiment-hf",
  "name": "Sentiment Analyzer (HuggingFace)",
  "version": "1.0.0",
  "provider_type": "huggingface",
  "description": "Multi-lingual sentiment via XLM-RoBERTa. Requires transformers + torch.",
  "language": ["en", "ne", "hi"],
  "behavior": "annotate",
  "annotations_key": "sentiment",
  "config": {
    "model": "cardiffnlp/twitter-xlm-roberta-base-sentiment",
    "task": "text-classification",
    "device": "cpu",
    "max_length": 512
  }
}
```

A.3 API provider — `manifests/sentiment_api.json`

```
{
  "module_id": "sentiment-api",
  "name": "Sentiment Analyzer (API)",
  "version": "1.0.0",
  "provider_type": "api",
  "description": "Sentiment via external REST API. Requires a running sentiment server.",
  "language": ["en", "ne"],
  "behavior": "annotate",
  "annotations_key": "sentiment",
  "config": {
    "endpoint": "http://localhost:5001/sentiment",
    "method": "POST",
    "headers": {},
    "timeout": 10,
    "response_annotations_field": "sentiment",
    "response_confidence_field": "score"
  }
}
```

SECTION B

Appendix B: Dependencies

Core runtime and development dependencies (Python 3.10+ required, uses `str | None` union syntax and `list[str]` generics).

```
pytest>=7.0
# Add module-specific deps here as needed
jsonschema>=4.0.0
requests>=2.28.0
flask>=3.0.0
```

Optional dependencies (documented, not in `requirements.txt`): `transformers` + `torch` for the HuggingFace adapter (≈ 2 GB install); the API adapter only needs `requests`.